

NAME(S):

Multi-branch if statements

Learning goals / relevant topics

- Conditionals (Runestone Ch. 4)
- Comparison Operators (Runestone Ch. 2)
- Functions (Runestone Ch. 5)
- Reeborg (<https://reeborg.ca/docs/en/reference/index.html>)

All of our work with conditionals thus far has only required `if` and `else` statements (where we only check whether a single condition is `True` or `False`). But in class, we've discussed `elif` statements, which can be used to distinguish between several conditional outcomes. For example, we can check whether a number `x` is `> 0`, `< 0`, or otherwise (`else`) equal to 0.

In this part of today's exercise, we're going to utilize `elif` statements to account for more than two possible outcomes. Recall that "elif" is a contraction of "else" and "if" and that many other programming languages require the use of `else if` instead of the contracted form. You can have as many `elif` branches as you need.

Walk Through Field

1. Download the Walk Through Field folder (or its contents) from the course website, which should contain 2 json files, which each define a single world for Reeborg.
2. Import both worlds into Reeborg and see what each world looks like.
3. Reeborg's goal is to walk along the bottom of the world and:
 - a. pick all strawberries
 - b. put any fallen stars back into the sky
 - c. perform a dance (turn 360 degrees) for each daisy

Here are before and after pictures for Field 0:



Figure 1: Field Before



Figure 2: Field After

- Write a program that accomplishes these goals for both worlds.

Hint: the `object_here()` function can be given a string argument to check if a specific type of object is present (e.g. `object_here("token")`).

Logical Operators

In Python, logical operators allow you to combine two conditions into a single, more complex condition. Python has three logical operators: `and`, `or`, and `not`.

`<condition> and <condition>` is True if both conditions are True. `<condition> or <condition>` is True if at least one of the conditions is True. `not <condition>` is True the condition is False.

- Download the Harvests folder (or its contents) from the course website and import each json file into Reeborg.
- Find `harvest1.py` from the folder and copy the code into Reeborg.
- Open the world Harvest 1 (from Reeborg's "introduction" worlds).
- Run the code from `harvest1.py` and make sure you understand what it's doing.
- Choose the world `harvest1a.json` (which you should have downloaded with the folder). You should notice that there are some holes in the field. Adapt the provided code so that Reeborg will harvest all of the carrots.

Note: the remaining portions also use worlds based on the downloaded files. Hint: you should only have to change a small portion of the code for this and each subsequent problem.

- Choose the world `harvest1b.json`. Adapt the code so that Reeborg will plant carrots in the empty spots of the field.
- Choose the world `harvest1c.json`. This time, Reeborg is carrying an infinite supply of carrots. Adapt the code so that Reeborg plants carrots in the gaps in the field.
- Choose the world `harvest5.json`. Reeborg's goal is now to pick all of the flowers (daisies and tulips).
- Choose the world `harvest7.json`. This time, Reeborg should pick everything except for the smiley tokens (Reeborg knows these as "token").
- Choose the world `harvest8.json`. This time, Reeborg should pick everything except for the flowers (daisies and tulips again).

Prompt: What is a benefit of being able to use multi-branch conditionals over simple "if" or "if..else.." statements? Give an example of how using a multi-branch statement may make your code more DRY.